

MCP and Tool-Integrated AI Applications

Linking LLMs to Functions, Files, APIs, CLIs, and External Systems

Tang Liang

DigiPen Institute of Technology Singapore

AI Application Development Bootcamp – Day 11

Day 11 Direction



Central question

How do we connect an LLM to real application capabilities without confusing the model, tool, protocol, controller, and business workflow?

Roadmap

- 1 From LLM to Application
- 2 Pipeline, Workflow, and Agent
- 3 Linking LLMs to Tools
- 4 MCP and Tool Integration
- 5 MCP Trade-offs and Security
- 6 Applications and Lab Preparation

Learning progression

Start with the complete application, compare integration choices, study MCP as one important standard, then apply the ideas in two lab scenarios.

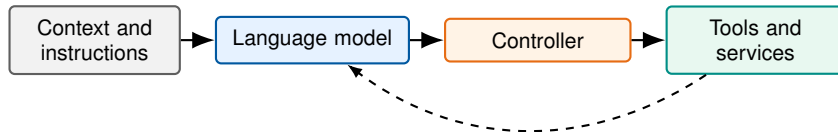
Why This Topic Matters for Employability

- Companies increasingly need graduates who can apply existing AI models inside useful software.
- The work is broader than prompt writing: applications need data, tools, state, security, testing, and user control.
- Valuable graduates can connect AI to existing code, services, engines, robots, databases, and workflows.
- This is applied AI engineering—not a course on training neural networks or deriving model mathematics.

Professional positioning

“I can turn an LLM into a controlled, testable application component.”

A Language Model Is Not the Whole Application



Engineering view

The model proposes or explains. Application code controls data access, tool execution, state changes, limits, and final delivery.

From Chatbot to Tool-Using System

Chat-only interaction

- Receives text or media
- Generates a response
- Relies mostly on supplied context
- Cannot directly change the external world

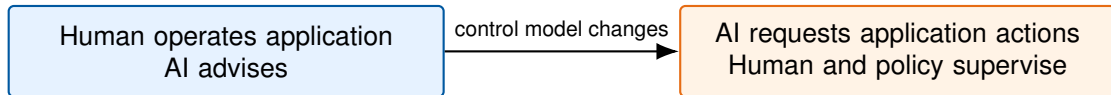
Tool-using application

- Reads current external data
- Calls deterministic functions
- Runs programs or services
- Produces controlled external effects

The shift

The system moves from generating language about an action to requesting an application to perform an action.

The Basic Mental Shift



Safe interpretation

Greater capability requires stronger workflows, permissions, validation, audit evidence, and human approval.

What Belongs in an Applied AI System?

Model

Context

Tools

State

Guardrails

- **Model:** interprets, drafts, classifies, plans, or selects an action.
- **Context:** supplies relevant instructions, facts, examples, and constraints.
- **Tools:** perform computation, retrieval, communication, or state change.
- **State:** records what has happened and what remains to be done.
- **Guardrails:** constrain inputs, outputs, actions, resources, time, and cost.

Deterministic Code and LLM Judgment Have Different Jobs

Task	Prefer deterministic software	Possible LLM contribution
Arithmetic	Exact calculation	Explain the result
Database lookup	Parameterized query	Interpret the request
Policy decision	Policy engine	Describe ambiguity or exception
Compiler/test	Build tools and assertions	Diagnose a failure
Report writing	Templates and validation	Draft readable narrative

Design principle

Use the LLM where language and ambiguity matter; use deterministic components where correctness is precisely defined.

A Tool Is a Typed Capability

Weak interface

```
process(data)
```

- Vague purpose
- Unclear input
- Unclear side effects
- Hard to validate

Stronger interface

```
calculate_average(readings)
```

- Numeric array input
- Defined failure cases
- Structured result
- No hidden side effect

```
{"count": 5, "sum": 137.0, "average": 27.4}
```

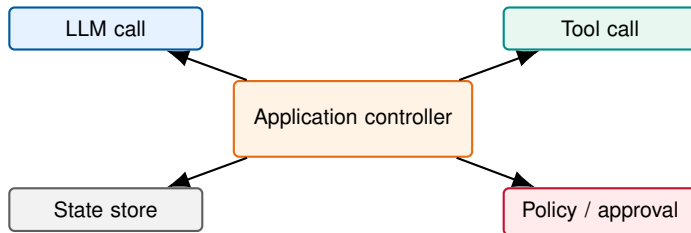
Tools, Applications, and Agents Are Not Synonyms

- A calculator is normally a **tool**, not an agent.
- A weather service is normally a **service/tool**, not an agent.
- A database is a **data system**, not an agent.
- A compiler is a **deterministic application**, not an agent.
- An agent/controller decides when and how to use capabilities in pursuit of a goal.

Remember

Tools are things an agent uses. Agents are controllers that decide how work progresses.

The Application Controller Owns Execution



- Validates model-produced arguments before execution.
- Routes calls to the correct implementation.
- Records results, errors, and approvals.
- Enforces timeout, step, scope, and cost limits.

A Practical Mental Model

- 1 **Prompting** gives instructions.
- 2 **Context and RAG** supply relevant information.
- 3 **Tools** provide external capabilities.
- 4 **MCP** standardizes access to tools and resources.
- 5 **Agents** coordinate dynamic actions and observations.
- 6 **Workflows** govern deterministic business processes.
- 7 **Humans and policies** govern sensitive decisions.

Day 11 focus

We concentrate on tool-integrated applications and MCP, while introducing enough agent architecture to understand why the integration exists.

Example: Game and Simulation Development

- Read a design specification
- Inspect engine logs
- Query asset metadata
- Generate a controlled configuration
- Run a build or simulation
- Summarize failures

Engineering boundary

The LLM should not silently replace authoritative assets, tests, physics calculations, or release decisions.

Employability signal

Integration, verification, and evidence are more valuable than a one-shot generated answer.

Example: Robotics and IoT



- Dedicated protocols carry real-time and high-volume signals.
- AI applications call bounded mission-level tools.
- Low-level motor and safety control remains deterministic.
- Human operators retain authority for consequential actions.

Section Summary: Think in Systems

- An LLM is one component, not the complete application.
- Tool interfaces should be explicit, typed, testable, and constrained.
- Deterministic code and LLM judgment should be assigned different responsibilities.
- The controller owns execution, state, limits, and evidence.
- “Agentic” describes how decisions and actions are coordinated, not merely the presence of an LLM.

Next

We need precise language for pipelines, workflows, and agent loops before choosing an integration technology.

2. Pipeline, Workflow, and Agent

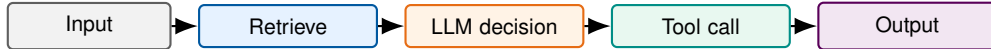
Why Terminology Becomes Confusing

- Product marketing often calls any tool-using LLM workflow an “agent.”
- Frameworks use the word for different runtime abstractions.
- Research traditions use stricter definitions involving autonomy and environments.
- Engineering teams still need to identify who chooses the next action and who owns state.

Course approach

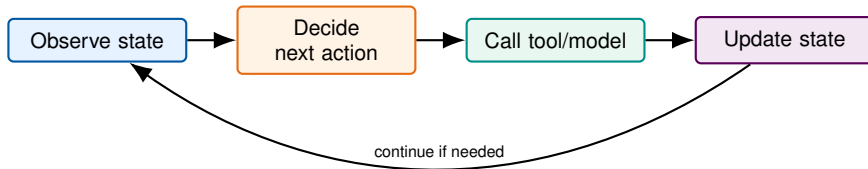
Recognize broad industry usage, then apply a stricter engineering definition when analysing control, safety, and testing.

Broad Industry Usage of “Agentic”



- The overall sequence can remain linear.
- The LLM may select a tool or interpret external information inside one stage.
- Many organisations call this an **agentic workflow**.

Stricter Engineering Definition



Defining feature

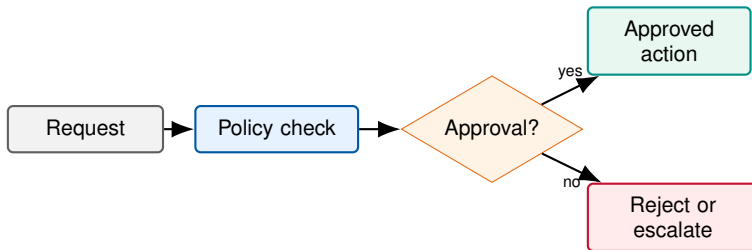
After observing a result, the controller can dynamically stop, retry, repair, escalate, or choose another action.

Pipeline: The Programmer Selects the Sequence



- Fixed order
- Predictable data flow
- Easy to test and reproduce
- May contain an LLM without being a dynamic agent

Workflow: Rules Govern States and Branches



- Explicit states, branches, permissions, approvals, and auditability
- The process remains authoritative even when an LLM assists inside a step

Agent Loop: The Next Action Can Change Dynamically

- 1 Observe current state and goal.
- 2 Select a model or tool action.
- 3 Execute under application control.
- 4 Validate and record the result.
- 5 Decide whether the goal is complete.

Possible decisions

- Stop successfully
- Retry
- Repair
- Ask the user
- Request approval
- Escalate as blocked

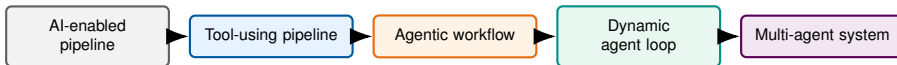
Dynamic Control Matters More Than a Circular Diagram

- A controller may stop after one successful iteration.
- A graph may contain branches without a visible cycle, yet still delegate a meaningful choice to an LLM.
- A repeated `while` loop using only fixed rules is traditional automation, not necessarily Agentic AI.
- A pipeline can contain a dynamic agent stage.

Architecture test

After receiving a new observation, who determines the next action: fixed program logic, an LLM/policy decision, or a human?

A Spectrum, Not a Binary Label



Use labels to explain architecture

The goal is not to win a terminology argument. The goal is to reveal control, state, risk, and verification responsibilities.

What Is an Agent Controller?

- Maintains task state and observations
- Selects or requests the next action
- Dispatches model and tool calls
- Applies policy and validation
- Handles errors and retries
- Records trace and audit evidence
- Enforces terminal conditions

Compact definition

An agent is a controller plus model/policy, instructions, tools, state, memory, and guardrails.

A Bounded Controller Skeleton

```
state = initialize_task()

for step in range(MAX_STEPS):
    observation = observe(state)
    action = choose_next_action(observation)
    checked_action = validate(action, policy)
    result = execute(checked_action, timeout=TIMEOUT)
    state = update(state, result)

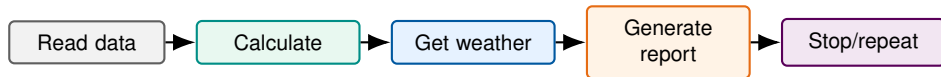
    if goal_reached(state):
        return success(state)
    if needs_human(state):
        return request_approval(state)

return blocked("step budget exhausted", state)
```

Why Bounds Are Part of the Architecture

Bound	Risk controlled
Maximum steps	Infinite loops and repeated tool calls
Timeout	Hung tools and unavailable services
Cost/token budget	Uncontrolled consumption
Retry limit	Repeated failure without progress
Tool scope	Unnecessary or unsafe actions
Human approval	Consequential or ambiguous operations

Day 11 Lab 1 on the Spectrum



- Core steps are selected by the programmer.
- The optional LLM generates language, not the numeric truth.
- Optional bounded repetition adds a loop but not necessarily dynamic planning.

Accurate description

A controlled tool-integrated pipeline with optional agent-loop behaviour.

Day 11 Lab 2 on the Spectrum

- The sales process has authoritative business states and policy checks.
- Tools retrieve customer, product, inventory, discount, and credit information.
- An LLM may interpret an exception or draft a summary.
- Approval and transaction creation remain workflow-controlled.

Preferred architecture

Workflow outside, agent inside.

Not a free agent

The LLM should not invent discounts, bypass credit rules, or create an order before required approval.

Section Summary: Identify the Control Structure

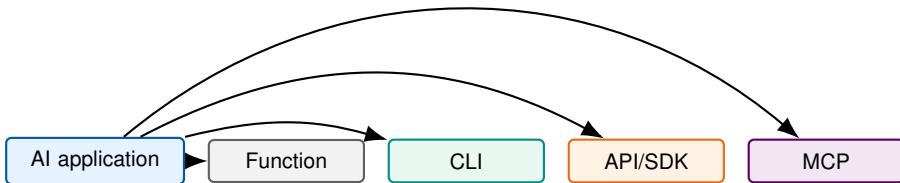
- Broad usage includes many LLM-assisted and tool-using workflows.
- The stricter definition requires dynamic control after observation.
- Pipelines optimize predictability; workflows govern states; agent loops handle uncertainty.
- Hybrid systems often combine all three.
- Bounds and terminal states are part of the design, not optional cleanup.

Next

Once the control structure is clear, choose how the application will reach its tools and services.

3. Linking LLMs to Tools

MCP Is One Integration Choice



Starting principle

Choose the simplest interface that meets reuse, security, deployment, and interoperability requirements.

Option 1: Direct Function Call

Strengths

- Lowest overhead
- Native types and exceptions
- Easy debugging
- No extra process or network

Limitations

- Coupled to one codebase/language
- Harder to reuse across hosts
- Application must implement dispatch
- Security boundary may be weak

```
result = calculate_average(readings)
```

Option 2: CLI and Subprocess



- Works with compilers, scripts, engine tools, test runners, and legacy programs.
- Language-independent at the process boundary.
- Structured JSON output can make the contract reliable.
- Requires safe argument handling, timeout, path control, and exit-code checking.

CLI Example: External Average Calculator

```
./tools/average_calculator numbers.txt
```

```
{  
  "count": 5,  
  "sum": 137.0,  
  "average": 27.4  
}
```

Why this is useful

The controller can treat an existing C++ program as a capability without rewriting its implementation in Python.

CLI Security Checklist

- Use an argument array; do not concatenate untrusted shell text.
- Allow only approved executable paths.
- Restrict working directories and input file paths.
- Set a timeout and output-size limit.
- Check exit code before trusting stdout.
- Treat stdout and stderr as untrusted input.
- Avoid exposing a general-purpose shell when a narrow executable is sufficient.

Common mistake

“The LLM generated the command” is not a reason to execute it without deterministic validation.

Option 3: Direct API or SDK

Suitable when

- A stable service already exists
- The application controls the integration
- Lowest practical latency matters
- Service-specific features are needed

Integration burden

- Different authentication
- Different request schemas
- Different errors and pagination
- Custom adapter code per service

REST and OpenAPI

- REST commonly exposes resources and operations over HTTP.
- OpenAPI can document endpoints, parameters, authentication, and schemas.
- An application may generate typed clients or tool definitions from an OpenAPI document.
- This is often sufficient when the service already has a good API.

Difference from MCP

OpenAPI describes a web API. MCP defines an AI-application protocol with lifecycle, capability discovery, tools, resources, prompts, and notifications.

Other Integration Technologies to Recognise

Technology	Typical role
gRPC / Protobuf	Typed, efficient service-to-service calls
GraphQL	Flexible structured data queries
WebSocket / SSE	Streaming or bidirectional updates
Queues / events	Durable asynchronous integration
SQL / DB client	Exact structured data access
Browser automation	Last resort when no suitable programmatic interface exists

Option 4: MCP

- Standardizes how AI applications discover and call capabilities.
- Allows a server to expose tools, resources, and prompts.
- Supports local stdio and remote HTTP-based connections.
- Can wrap functions, CLIs, APIs, SDKs, databases, or domain services.
- Decouples the AI host from many implementation details.

Value proposition

Write or deploy an adapter once, then make the capability understandable to multiple compatible AI hosts.

These Technologies Operate at Different Layers

Skills: reusable task instructions, checklists, scripts, and templates

MCP: standardized AI application-to-tool/resource interface

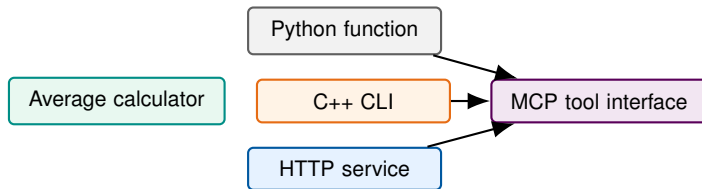
REST/SDK: application or service API

CLI/function: local implementation interface

Do not force a false competition

A skill can instruct an agent to use an MCP tool whose server calls a REST API or CLI.

One Capability, Several Wrapping Choices



Abstraction boundary

The caller sees the tool schema and structured result; the server owns the implementation choice.

Decision Guide

- 1 Is the capability already in the same application? Start with a function.
- 2 Is it an existing local executable? Start with a safe CLI wrapper.
- 3 Is it an established remote service? Start with its API or SDK.
- 4 Must different AI hosts reuse and discover it? Consider MCP.
- 5 Is the remote party an independent agent with its own task lifecycle?
Consider A2A later.

Architecture habit

Do not add a protocol merely because it is fashionable; add it when it solves a boundary or reuse problem.

Employability: Explain the Trade-off

An effective interview answer is not:

"I use MCP for everything."

It is:

"I used a direct function for in-process logic, wrapped an existing C++ executable through a constrained CLI, and exposed selected capabilities through MCP because multiple AI hosts needed a stable tool contract."

Signal

The choice demonstrates interface design, reuse, security awareness, and operational judgment.

Section Summary: Use the Simplest Suitable Boundary

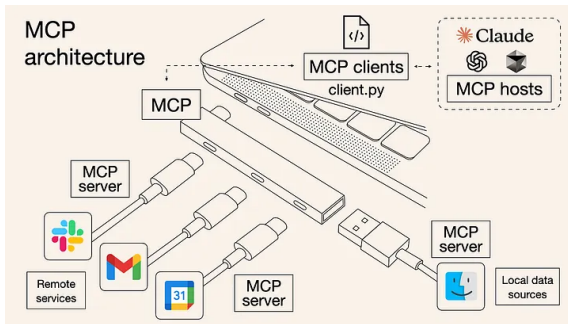
- Direct functions minimize overhead inside one application.
- CLI is a powerful, model-independent bridge to existing tools.
- APIs and SDKs are often best for established remote services.
- MCP adds a standardized AI-facing interface and discovery mechanism.
- Open source, public API, and interoperable standard are different properties.

Next

We now examine MCP precisely: architecture, lifecycle, primitives, messages, and adapters.

4. MCP and Tool Integration

MCP: The Universal-Adapter Idea

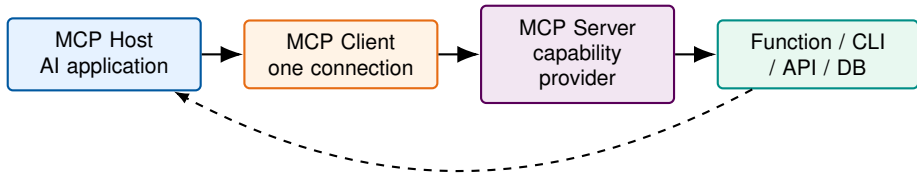


- AI hosts need access to many tools and data sources.
- MCP provides a common interface between hosts and servers.
- Servers can adapt existing services rather than replacing them.

First mental model

MCP is an adapter standard for AI applications.

The Canonical Architecture



Important detail

The host coordinates the AI experience and commonly creates a separate MCP client for each server connection.

What Problem Does MCP Solve?

Without a common interface:

- every host implements a custom connector for every service;
- tool names, descriptions, argument schemas, and results vary;
- authentication and deployment patterns are repeated;
- integrations become difficult to reuse across AI applications.

MCP contribution

A standard protocol for exchanging context and invoking declared capabilities between AI applications and servers.

MCP Does Not Replace Underlying Protocols

Underlying technology	MCP relationship
REST API	Server adapts API operations into tools/resources
CLI executable	Server launches and validates the executable
SQL/database	Server exposes controlled queries or domain operations
ROS/ROS2	Server exposes mission-level robot services
MQTT	Server may configure or query IoT operations
Video/WebRTC	MCP may control or return a stream reference, not carry raw video

Protocol layering: MCP operates at the AI application integration layer; dedicated protocols continue to perform their original jobs.

MCP Has a Data Layer and a Transport Layer

Data layer

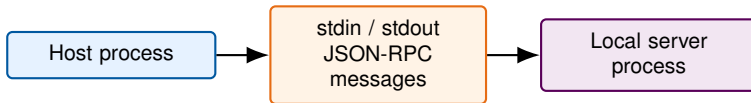
- JSON-RPC messages
- Lifecycle
- Tools
- Resources
- Prompts
- Notifications

Transport layer

- stdio
- Streamable HTTP
- Connection handling
- Message framing
- Authentication support

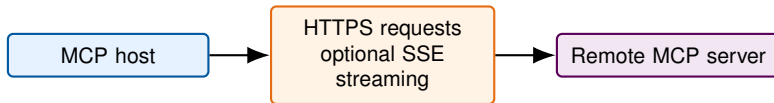
Source: Model Context Protocol, Architecture Overview

Local MCP: stdio Transport



- Host commonly launches the server process.
- Messages travel through standard input/output.
- Diagnostic logging should use standard error, not corrupt stdout messages.
- Credentials should normally come from the environment or local configuration.

Remote MCP: Streamable HTTP



- Suitable for remotely deployed and multi-client services.
- Requires serious authentication, authorization, origin, and network controls.
- Remote servers should be treated as external trust boundaries.

Lifecycle: Connect Before Calling Tools

- 1 Client sends `initialize` with protocol version and capabilities.
- 2 Server returns its supported capabilities and identity information.
- 3 Client sends an initialized notification.
- 4 Client discovers available primitives.
- 5 Requests and notifications proceed under the negotiated connection.
- 6 Connection eventually closes or terminates on incompatibility/error.

Why lifecycle matters

Both sides need to know what the other supports before attempting operations.

Initialization Is Capability Negotiation

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {},
    "clientInfo": {"name": "lab-host", "version": "1.0"}
  }
}
```

Do not memorize the JSON

Understand the purpose: version compatibility, capability discovery, and identity exchange.

Server Primitive 1: Tools

- Executable functions that an AI application can request.
- Described by name, purpose, and input schema.
- May optionally define structured output.
- Can retrieve information or cause side effects.

```
calculate_average_external(path: string)
-> {count: integer, sum: number, average: number}
```

Safety distinction

Reading a temperature file and deleting a project directory are both “tools,” but they require very different permissions and approval.

Server Primitive 2: Resources

- Data sources provided as contextual information.
- Examples: file contents, schemas, records, manuals, configuration, or generated artifacts.
- Addressed using resource identifiers.
- Applications decide how and when to place resource content into model context.

Tool or resource?

Use a resource to expose information; use a tool when computation or action is required.

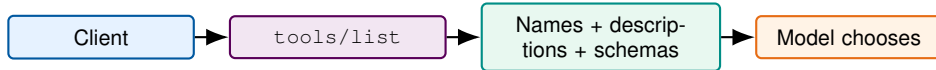
Server Primitive 3: Prompts

- Reusable interaction templates supplied by a server.
- Can encode domain-specific usage guidance and examples.
- May accept parameters.
- Usually selected by the user/application rather than autonomously executed as an external action.

Different from a skill

A prompt is an MCP primitive; a skill is a broader reusable workflow package that may contain instructions, scripts, templates, and references.

Tool Discovery Before Execution



Dynamic catalogue

A server can change its available tools and notify connected clients that the list has changed.

JSON-RPC Request, Response, Notification

Request

Has `id`; expects a response.

```
{"id":2,  
  "method":"tools/list"}
```

Response

Repeats `id`; returns result or error.

```
{"id":2,  
  "result":{...}}
```

Notification

No `id`; no response expected.

```
{"method":  
  "notifications/..."}  
}
```

Lab Example: Calling the External Calculator

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "calculate_average_external",
    "arguments": {"path": "numbers.txt"}
  }
}
```

Routing

The MCP server validates the arguments, launches the approved C++ executable, parses its JSON, and returns a structured result.

Lab Example: Structured Tool Result

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "result": {
    "content": [{"type": "text", "text": "Average: 27.4"}],
    "structuredContent": {
      "count": 5,
      "sum": 137.0,
      "average": 27.4
    },
    "isError": false
  }
}
```

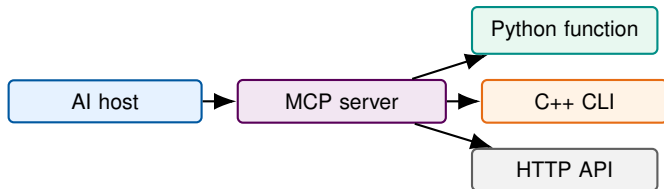
Why Structured Results Matter

- Application code can validate fields and types.
- The result can be consumed without parsing natural-language prose.
- Tests can compare expected and actual values.
- Another UI or service can reuse the result.
- The LLM can receive a concise, reliable representation.

Design habit

Return machine-readable truth first; generate human-readable explanation separately.

MCP Server as Adapter



Separation of concerns

The server translates a stable tool contract into implementation-specific calls and errors.

Control Plane and Data Plane

Control plane: discovery, commands, configuration, approvals, status, errors, audit

Data plane: video, LiDAR, telemetry, audio, high-rate sensor streams, bulk data

- MCP is usually best suited to control and context exchange.
- Dedicated protocols should carry high-throughput or hard-real-time data.
- A tool may start a stream or return a stream reference without carrying every frame.

MCP Does Not Make a System an Agent

- MCP supplies a protocol for capabilities and context.
- A fixed application can call MCP tools without any LLM.
- An LLM can call an MCP tool inside a fixed pipeline.
- A dynamic agent loop can use functions, CLI, APIs, MCP, or all of them.

Separate the axes

Integration protocol answers “how can the application reach a capability?” Agent architecture answers “who decides what happens next?”

Section Summary: Understand, Do Not Memorize

- Host, client, server, and underlying implementation have different roles.
- MCP negotiates capabilities before use.
- Tools act, resources provide data, and prompts provide reusable templates.
- JSON-RPC supplies request, response, and notification envelopes.
- MCP servers commonly adapt existing functions, CLIs, APIs, and databases.
- Dedicated protocols still carry real-time and high-volume data.

5. MCP Trade-offs and Security

Is MCP Always the Right Choice?

- MCP adds another component, dependency, and failure boundary.
- Tool descriptions and results consume context and tokens.
- Large catalogues can make tool selection harder.
- Remote servers create authentication and supply-chain concerns.
- A direct function, CLI, or API may be faster and simpler.

Engineering conclusion

MCP is useful when interoperability and reuse justify the additional layer. It is not mandatory for every AI application.

The “MCP Is Dead” Argument

The provocative argument highlights real concerns:

- extra complexity compared with calling an API directly;
- maintenance and configuration burden;
- possible performance and usability costs;
- security risks from broad or poorly understood integrations.

How to read such claims

Treat them as design criticism, not as evidence that an actively governed and widely implemented protocol has disappeared.

Source: Nick Babich, “MCP is Dead,” UX Planet, 6 April 2026: <https://uxplanet.org/mcp-is-dead-cf16b667ba6d>

MCP Is Active—But Hype Is Not a Design Reason

- MCP is governed under the Linux Foundation's Agentic AI Foundation.
- Major AI ecosystems support or integrate MCP capabilities.
- Active adoption does not imply that every application should use it.

Balanced position

Learn MCP because it is relevant and useful. Choose it because requirements justify it, not because it is fashionable.

Source: Linux Foundation, Agentic AI Foundation announcement:

<https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>

Choose the Simplest Suitable Integration

Situation	Preferred starting point
Logic inside the same application	Direct function
Existing local program or compiler	CLI/subprocess
Established remote service	Direct API or SDK
Reusable capabilities across AI hosts	Consider MCP
Independent agent collaboration	Consider A2A on Day 12

Rule: Standardization has value, but simplicity also has value.

Cost Is More Than Network Latency

- **Engineering cost:** server implementation, configuration, upgrades, tests.
- **Operational cost:** processes, hosting, authentication, monitoring.
- **Model cost:** tool schemas and results consume context/tokens.
- **Interaction cost:** approvals and errors may add steps.
- **Security cost:** additional trust boundary and permission design.

Benefit side

Reusable discovery, consistent contracts, cross-host integration, and cleaner adapter boundaries may repay those costs.

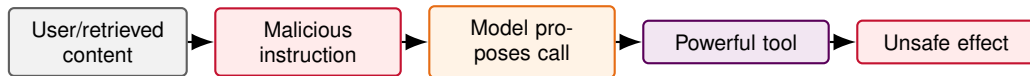
Large Tool Catalogues Can Reduce Reliability

- Similar tool names can confuse selection.
- Long descriptions increase context size.
- Broad capabilities increase the attack surface.
- Irrelevant tools distract the model from the current task.

Mitigations

Expose only task-relevant tools, use precise names/descriptions, group by server/domain, and load tool definitions selectively.

How Unintentional Tool Requests Happen



- Prompt injection
- Poisoned documents or web pages
- Manipulated tool results
- Ambiguous user intent
- Excessive permissions

Treat Every Boundary as Untrusted

- Validate user input before it reaches a tool.
- Validate model-generated arguments against schemas and policy.
- Treat server descriptions and tool results as untrusted content.
- Do not assume an installed server is safe because it implements MCP.
- Sanitize data before including it in logs or model context.

Protocol versus trust

A standardized message format does not prove that the sender, server, tool, or result is trustworthy.

Least Privilege for Tool-Using Applications

- Give the process only the filesystem paths it needs.
- Separate read tools from write tools.
- Avoid general shell access when a narrow operation is sufficient.
- Use scoped credentials for remote services.
- Disable unnecessary tools for the current task.
- Run risky operations in an isolated workspace or sandbox.

Day 12 extension

Permissions can also be granted just in time, limited to one task, and automatically revoked after use.

Confirmation Must Show the Real Action

Weak confirmation:

“Allow the agent to continue?”

Stronger confirmation:

“Create order Q001 for SGD 6,480, reserve four units of P100, and append an audit event?”

- Show exact arguments and affected resources.
- Distinguish reversible and irreversible effects.
- Revalidate permissions immediately before execution.

Timeouts and Budgets Are Security Controls

Control	What it prevents
Tool timeout	Hung process or unavailable service
Maximum retries	Endless repeated failure
Maximum steps	Runaway agent loop
Output-size limit	Memory/context exhaustion
Rate limit	Service abuse and accidental flood
Cost/token budget	Unexpected model expenditure

Audit Evidence for Every Significant Call

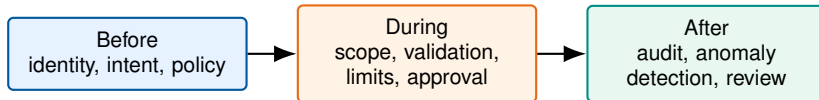
Record:

- run and user identity;
- tool/server name;
- validated arguments or protected hash/reference;
- permission and approval decision;
- start/end time and status;
- structured result category;
- retry, error, and escalation information.

Privacy warning

Logs can contain sensitive prompts, tool arguments, and results. Minimize, redact, protect, and retain them deliberately.

Security Before, During, and After



Defence in depth

No single intent classifier, prompt rule, schema, approval dialog, or log is sufficient by itself.

Section Summary: Interoperability with Judgment

- MCP is active and useful, but not automatically efficient or secure.
- CLI and direct APIs remain important alternatives.
- Tool catalogues should be narrow and task-relevant.
- Validate every boundary and apply least privilege.
- Confirm consequential actions using exact effects.
- Enforce time, step, retry, output, and cost limits.
- Preserve audit evidence without carelessly retaining sensitive data.

6. Applications and Lab Preparation

Two Lab Questions, Different Expectations

Question 1: Required

- IoT Temperature Application
- Detailed guided procedure
- Provided working components
- Complete during the bootcamp

Question 2: Optional

- Sales Transaction Application
- Open-ended design
- Starter data/specifications
- Continue after class if desired

Question 1 Scenario: IoT Temperature Application

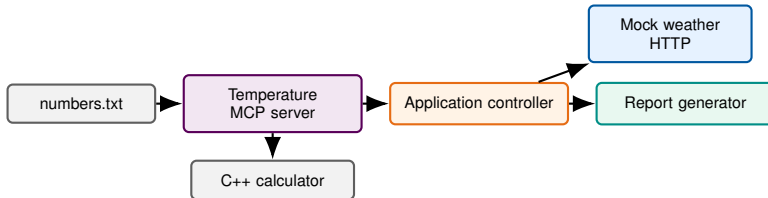
An IoT device records historical indoor temperatures in `numbers.txt`. The application:

- 1 reads the historical values through an MCP tool;
- 2 calculates the average;
- 3 obtains mock current Singapore temperature over HTTP;
- 4 generates a comparison report;
- 5 recommends whether air-conditioning should be enabled;
- 6 optionally repeats within configured limits.

Stable classroom environment

The weather service uses mock data so the exercise does not depend on Internet availability or a paid external API.

Question 1 Architecture



Controller responsibility

Coordinate MCP calls, HTTP access, report generation, configuration, errors, and exit conditions.

Question 1 File Map

```
lab1_temperature_application/  
  agent/agent_controller.py  
  mcp_servers/temperature_mcp_server.py  
  weather_server/mock_weather_server.py  
  cpp/average_calculator.cpp  
  tools/average_calculator[.exe]  
  report_generators/  
  numbers.txt  
  config.json  
  requirements.txt  
  README.md
```

Role clarity

`agent_controller.py` coordinates the application. The MCP server exposes capabilities; it is not the complete agent/controller.

Internal and External Calculators Are Alternatives

Internal MCP tool

The server directly calls a Python calculation function.

External-app wrapper

The server launches the compiled C++ calculator and parses its JSON output.

Do not draw them as consecutive stages

They are two implementations of one logical average-calculation capability. Level 1 uses Python; Level 2 and later use C++.

Question 1 Progressive Levels

- 1 Internal Python average tool + deterministic report
- 2 External C++ calculator through MCP + deterministic report
- 3 Simulated LLM-compatible report generator
- 4 Optional real LLM report through an available CLI/provider
- 5 Optional bounded repeat mode

Core assessment

Understanding the boundaries, data flow, tool contract, structured result, and failure handling matters more than reaching the optional real-LLM level.

What the LLM Should and Should Not Do

Task	Preferred component
Read file	MCP tool
Calculate average	Python/C++ deterministic tool
Retrieve current mock temperature	HTTP service
Compare exact values	Application code
Draft readable advice	Template, simulated LLM, or optional real LLM
Enforce loop exit	Controller

Question 1 Failure Tests

At minimum, inspect what happens when:

- `numbers.txt` is missing;
- a line is not numeric;
- the file is empty;
- the executable path is wrong;
- the executable returns invalid JSON;
- the weather server is unavailable;
- a file path escapes the permitted project directory;
- the loop reaches its maximum iteration count.

Professional habit

A successful demo is incomplete evidence. Failure handling reveals the quality of the integration.

Question 1 Required Evidence

- 1 Correct Python environment and dependencies
- 2 Successful standalone C++ executable result
- 3 Successful mock weather response
- 4 Level 1 application output
- 5 Level 2 application output
- 6 Level 3 simulated-LLM output
- 7 At least two failure-case observations
- 8 Short explanation of internal versus external implementation

Optional evidence

Real LLM mode and bounded repeat mode if available and time permits.

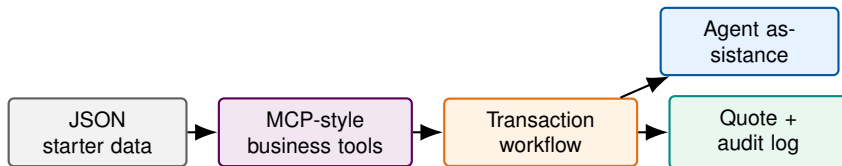
Question 2 Scenario: Simulated Sales Transaction

The application processes an order request by retrieving:

- customer information;
- product details;
- inventory;
- discount policy;
- credit status;
- approval requirements.

It produces a quote decision and audit evidence. It does not connect to real payment, CRM, ERP, email, or personal data.

Question 2 Suggested Architecture



Authority boundary

The workflow owns official state transitions, calculations, approvals, and simulated order creation.

Question 2 Minimum Tool Families

Read and calculate

- get order request
- get customer/product
- check inventory
- calculate quote
- apply discount
- check credit limit

State and evidence

- create quote
- request approval
- create simulated order
- reserve simulated inventory
- write audit event

Question 2 Workflow Outside, Agent Inside



- Agent may interpret ambiguous text, explain decisions, or draft summaries.
- Policy code validates discounts, credit, inventory, and approval.
- No state-changing action occurs outside the allowed workflow.

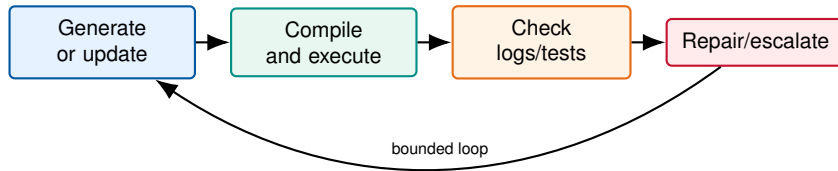
Question 2 Verification Cases

- 1 Normal quote can be created
- 2 Insufficient inventory
- 3 Invalid customer ID
- 4 Invalid product ID
- 5 Zero or negative quantity
- 6 Credit limit exceeded
- 7 Approval required by threshold
- 8 Audit record written for every significant state change

Optional activity

Question 2 is open-ended and does not require bootcamp submission. Use it to extend the architecture after completing Question 1.

Day 12 Preview: From Integration to Loop Engineering



- State, memory, and database design
- Dynamic permissions and semantic layers
- Sub-agents, multi-agent systems, and A2A
- Verification and repair loops
- Agentic Digital Twin scenario generation

Final Checklist

Can you now explain:

- why an LLM is not the complete application?
- the difference between pipeline, workflow, and agent loop?
- when to use a direct function, CLI, API, or MCP?
- host, client, server, tool, resource, and prompt roles?
- why MCP may add cost or risk?
- how least privilege, validation, limits, approval, and audit work together?
- why Lab Question 1 is mainly a controlled pipeline?

Final Takeaway

MCP connects AI applications to controlled tools and resources. It is one integration option alongside direct functions, CLI, and APIs. The application controller remains responsible for state, validation, permissions, execution limits, audit evidence, and human approval.

Preparation for Day 12

Tool access provides capability. Loop engineering determines how capability is coordinated, constrained, remembered, and verified.

Selected References and Further Reading

- Model Context Protocol, Introduction:
<https://modelcontextprotocol.io/docs/getting-started/intro>
- Model Context Protocol, Architecture:
<https://modelcontextprotocol.io/docs/learn/architecture>
- Model Context Protocol, Server Concepts:
<https://modelcontextprotocol.io/docs/learn/server-concepts>
- Model Context Protocol Specification:
<https://modelcontextprotocol.io/specification/2025-11-25>
- JSON-RPC 2.0 Specification: <https://www.jsonrpc.org/specification>
- Linux Foundation, Agentic AI Foundation announcement: <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>
- Anthropic, Building Effective Agents:
<https://www.anthropic.com/research/building-effective-agents>
- OpenAI Agents SDK documentation: <https://openai.github.io/openai-agents-python/>